

Complemento de Dois*

Juliano Mourão Vieira

18 de abril de 2025

Sumário

1	TRÊS POSSIBILIDADES DE CÁLCULO	1
1.1	CIRCUITARIA	1
1.2	COMPLEMENTO PARA A POTÊNCIA DE 2	2
1.3	PESO NEGATIVO NO MSB	2
1.4	EXEMPLO	2
2	EXPLICAÇÃO	3
2.1	CARRY E OVERFLOW	4
2.1.1	Unsigned	4
2.1.2	Signed	4
2.1.3	Misturado!	4
2.2	EXEMPLOS DETALHADOS	6
2.3	ASSEMBLY E COMPLEMENTO DE 2	7
3	A OBSOLETA SINAL-MAGNITUDE	9
4	EXERCÍCIOS	9

1 TRÊS POSSIBILIDADES DE CÁLCULO

Vou pular aqui as intuições iniciais a respeito de complemento de 2, como o odômetro de um carro ou trabalhar com excesso-n (bias de n). Consulte um bom livro de Eletrônica Digital se precisar.

1.1 CIRCUITARIA

O que é normalmente aprendido em Sistemas Digitais é: para representar um número -x em complemento de 2, faça o binário do x, inverta todos os

*rev3

bits (isso é o complemento de 1) e some 1. Isso gera um circuito simples e rápido, embora seja meio chato de fazer na mão.

Exemplo: como representar -7 em complemento de 2 com 8 bits? $7 = 111_2 = 00000111_2$; invertendo e somando 1, temos $11111000_2 + 00000001_2 = 11111001_2 = -7$.

Note que *sempre que trabalhamos em complemento de 2 é necessário identificar todos os bits do número e a largura da palavra deve ser indicada*. Isso se dá porque, de outra forma, pode ser ambíguo saber se um número é negativo ou não. O número 111111_2 é negativo? Se estiver em complemento de 2 com 6 bits, é, senão não é.

1.2 COMPLEMENTO PARA A POTÊNCIA DE 2

Uma forma simples e rápida de calcular no olho é subtrair o número da próxima potência de 2, e o binário do resultado equivale ao negativo em complemento de 2.

Voltando ao nosso exemplo: trabalhamos com 8 bits, então a potência é $2^8=256$. Para calcular o binário de -7, fazemos $256-7=249$, e o binário de $249=11111001_2$, que é o mesmo número que obtivemos anteriormente. Portanto -7 em complemento de 2 de 8 bits é isomórfico a 249 binário.

1.3 PESO NEGATIVO NO MSB

Outra forma simples de olhar para um número em complemento de 2 é simplesmente inverter o sinal do peso do bit mais à esquerda. Veja a conversão do número -7:

$$11111001_2 = 1 * (-128) + 1 * 64 + 1 * 32 + 1 * 16 + 1 * 8 + 0 * 4 + 0 * 2 + 1 * 1 = -128 + 64 + 32 + 16 + 8 + 1 = -7$$

Para fazer a conta no outro sentido, basta “descontar”: se o MSB vale -128, então $-128 + \text{resto} = -7$, então o $\text{resto} = +121 = 1111001_2$, completando os sete bits à direita.

1.4 EXEMPLO

Como representar o número -78 em complemento de 2 de 8 bits?

- $78_{10} = 01001110_2$; invertamos e incrementamos para $10110001_2 + 1 = 10110010_{\text{Compl}2}$;
- Perceba: $10110010_{\text{Compl}2} = 1 * (-2^7) + 0 * 2^6 + 1 * 2^5 + 1 * 2^4 + 0 * 2^3 + 0 * 2^2 + 1 * 2^1 + 0 * 2^0 = -128 + 32 + 16 + 2 = -78_{10}$;
- E ainda: a próxima potência é $2^8 = 256$; calculamos $256 - 78 = 178$.

Observe que $178_{10} = 10110010_2$, que são os mesmos bits determinados antes.

2 EXPLICAÇÃO

A ideia é simples: como temos um número limitado de n bits num processador, só podemos representar 2^n números; todos os números que estouram estes limites são representados em **módulo 2^n** (!)¹

Portanto um número a de n bits representa, na verdade, vários números b possíveis, caracterizando uma congruência:²

$$b \equiv k * 2^n + a \quad (1)$$

Por exemplo, novamente em 8 bits, se $a = 00110011_2$ (esse binário dá 51 em decimal), $b = k * 256 + 51$:

k	b
-1	-205
0	51
1	307
2	563
3	819

Ou seja, se estamos limitados a 8 bits, a sequência 00110011_2 pode indicar as grandezas -205, 51, 307,...³ Claro que está faltando bit para os números maiores: $819 = \underline{11}00110011_2$, de 10 bits; não cabe em 8.

Agora vamos fazer o mesmo para o -7 em 8 bits, que é 11111001_2 :

k	b
-2	-263
-1	-7
0	249
1	506
2	762

Isso levanta a questão: se um registrador do hardware está com os bits 11111001_2 , qual é o número representado? E a resposta é: *depende do que o programa fez e do que o programador quer dizer*.

A linguagem C⁴ define explicitamente duas possibilidades para qualquer variável integral: *signed* e *unsigned*, ou seja, sinalizado e não sinalizado.

¹Também dá pra usar a notação $\text{mod } 2^n$, mais familiar na Ciência da Computação.

²Se alguém quiser, tem uma álgebra mais sofisticada embasando essas coisas; grupos cíclicos, parece. Eu acho lindo, mas tá fora do meu alcance.

³Isso é análogo aos ângulos no círculo: $45^\circ \equiv -315^\circ \equiv 405^\circ$ porque $\alpha = k * 360^\circ + \beta$.

⁴C é nosso default em Microcontroladores e Sistemas Embarcados, embora C++ tenha uma fatia crescente e seja uma expansão do C standard.

Com isso, o *programador* pode definir o que deseja com certo conjunto de bits. Portanto, se a variável é *unsigned*, então 11110001_2 representa o valor decimal 249; se a variável é *signed*, então 11110001_2 está em complemento de 2 e portanto vale -7.

Mas isso obrigatoriamente só vale se não houve nenhum estouro em uma operação anterior...

2.1 CARRY E OVERFLOW

Todos os processadores modernos possuem indicadores (*flags*) de quando uma operação aritmética excede a capacidade de bits dos seus registradores. A ideia é bem simples: quando há estouro *unsigned*, isso é um *carry*; quando há um estouro *signed*, é um *overflow*.

Note que se a operação é *unsigned*, o *carry* é a *flag* relevante e o *overflow* pode ser desprezado; de forma simétrica, só atentamos ao *overflow* para operações sinalizadas.

2.1.1 Unsigned

Digamos que precisamos somar $120+80$ em 8 bits, tudo sem sinal. O resultado é 200, não há *carry*. Se somarmos $200+200$, por outro lado, o número 400 *não cabe* em 8 bits, então há *carry* (é equivalente ao bit de vai-um). O resultado final no registrador é 144 (pois o bit extra, o “nono bit”, vale 256; $144+256=400$).

A subtração é similar: $130-14=116$, sem *carry*. Agora, $20-40=236$, que equivale a -20 em complemento de 2. Se o resultado é negativo, há *carry* (afinal, é um estouro de capacidade).

2.1.2 Signed

Novamente em 8 bits, se somarmos $34+(-53)$ não há *overflow*: -19 é representável tranquilamente. Mas se somarmos $105+102$, o resultado 207 *não é representável*, portanto há sim *overflow*. Isso pode ser verificado quando os operandos têm o mesmo sinal (ambos positivos ou ambos negativos) mas o resultado da soma tem o sinal invertido ao deles.

Os limites que devem ser obedecidos são: $-2^{n-1} \leq \text{resultado} < 2^{n-1}$. Portanto, se qualquer operação em 8 bits exceder -128 ou +127, há estouro.

2.1.3 Misturado!

Não é uma boa ideia misturar *signed* e *unsigned* na mesma expressão: eles possuem limites diferentes e nem sempre o compilador saberá como lidar com eles. O resultado final pode depender da linguagem, do compilador e

até do assembly do processador.⁵

Em C, caso uma operação ocorra entre um *signed int* e um *unsigned int* (32 bits nos PC's atuais) o *signed int* será convertido em um *unsigned*. No código abaixo:

```
#include <stdio.h>
int main(void) {
    signed    int s_int=-1000;
    unsigned int u_int=3000111222;
    long long  l_int;
    char       teste1, teste2;

    l_int = u_int+s_int;
    teste1 = (s_int<u_int);
    teste2 = (s_int>u_int);
    printf("%lld\n%d\n%d\n", l_int, teste1, teste2);
    return 0;
}
```

O resultado da compilação⁶ e execução é:

```
> ./main
3000110222
0
1
```

Observe os três resultados, em ordem:

- A soma na variável `l_int` foi feita corretamente. O que ocorreu foi que o número -1000 foi convertido de *signed* para *unsigned* (virou 4 294 966 296, que é $2^{32}-1000$). Em 32 bits isso significa usar módulo 2^{32} , ou seja, $(u_int-1000)$ e $(u_int+4294966296)$ produzem o mesmo resultado, 3000110222.
- A comparação para `teste1` resultou *falso!* Apesar de que até para uma criança seja óbvio que $-1000 < 3000111222$, não foi essa a interpretação, o que é contraintuitivo. A comparação percebe que estamos misturando dois int's diferentes, promove o *signed* para *unsigned*, o que transforma o -1000 em $2^{32}-1000 = 4294966296$. A comparação que ocorre agora é `teste1 = (4294966296 < 3000111222)`, o que é falso.

⁵Diretamente do Kernighan e Ritchie, "The C Programming Language" seção A.6.5: "Unexpected results may still occur when an unsigned expression is compared to a signed expression of the same size." Ou seja, soluções para isso não têm portabilidade em C90, embora as regras cubram a maior parte dos casos.

⁶Usei o Compiler Explorer (<https://godbolt.org/>), com o clang version 7.0.0-3~ubuntu0.18.04.1 (tags/RELEASE_700/final)

- Consistente com isso, teste2 resulta em verdadeiro pelo exato mesmo motivo.

Ao trabalhar com microcontroladores de baixo custo, estas considerações sobre números negativos eventualmente acabam tendo de ocorrer.

2.2 EXEMPLOS DETALHADOS

Dada a conta: $11001110_2 + 11000111_2$ com resultado num registrador de 8 bits, quais são os valores decimais envolvidos? Houve estouro da capacidade de 8 bits ou não? Qual o resultado final?

Chamemos o valor 11001110_2 de x e o valor 11000111_2 de y . Temos quatro possibilidades interpretativas, usando notação da linguagem C:

tipo	x	y
<i>signed char</i>	-50	-57
<i>unsigned char</i>	206	199

O resultado final da soma será 10010101_2 , independentemente de haver sinal ou não, pois a ULA faz as somas sem saber se o número é sinalizado ou não, pois o circuito e a operação matemática são os mesmos. Este valor 10010101_2 tanto pode ser 149, caso o resultado seja interpretado como *unsigned*, como pode ser -107, se for *signed*.

Perceba que *o resultado final está correto*, exceto quando tanto x quanto y são *unsigned*:

- $-50 - 57 = -107$
- $-50 + 199 = 149$
- $206 - 57 = 149$
- $206 + 199 = 405$, e $405 = 256 + 149$ (o 256 é o *carry*)

Isso ocorre porque as contas de 8 bits são feitas em *módulo 256*.

Observe que, nesta conta $11001110_2 + 11000111_2$, há *carry* e não há *overflow*. O *overflow* só é relevante para o caso 1 e o *carry* é feito para o caso 4. Os casos 2 e 3 são “perigosos” para o programador, misturando representações sinalizada e não sinalizada; o resultado dá certo por causa do módulo 256 nas operações.

Mais um exemplo: veja as 4 interpretações de uma soma de 8 bits entre os números 130 (cujo complemento é -126) e 101 (que vale 101 tanto em *signed* quanto em *unsigned*):

unsigned	130	signed	-126	signed	-126	unsigned	130
unsigned	$+ 101$	signed	$+ 101$	unsigned	$+ 101$	signed	$+ 101$
	231		-25		-25		231
(a)		(b)		(c)		(d)	

Como se pode imaginar, $256-25=234$. Não há *carry*, não há *overflow*.

2.3 ASSEMBLY E COMPLEMENTO DE 2

Por conta das peculiaridades com a sinalização, algumas operações precisam ser adaptadas de acordo.

a. Negação do maior negativo

O circuito para negação só inverte os bits e soma 1; portanto, $-(-120) = -(10001000_2) = 01110111+1 = 01111000_2 = +120$, em 8 bits. Mas o que ocorre ao calcular $-(-128)$ em 8 bits?

$$-(-128) = -(10000000_2) = 01111111+1 = 10000000_2 = -128!!!$$

Isso faz sentido, pois em 8 bits os números estão entre -128 e +127, inclusive. Portanto a *negação pode produzir um resultado incorreto* se o operando for o menor negativo possível.

b. Comparações

O programa em C anterior já deve ter colocado o terror algébrico adequado no leitor. Então não surpreende que necessitemos de comparações diferenciadas para *signed* e *unsigned*.

Em 8 bits, como de hábito, se quisermos comparar 11111111_2 com 00000001_2 , a questão é: estamos comparando sinalizados -1 com +1 ou estamos comparando não-sinalizados +127 com +1? Se formos comparar $11111111_2 < 00000001_2$, podemos ter $(-1 < +1 \Rightarrow \text{true})$ e $(+127 < +1 \Rightarrow \text{false})$.

No assembly RISC-V, temos diferenciação: há a instrução *slt* (set if less than) que compara com sinal (seria -1 e +1 no exemplo) e a instrução *sltu* que compara sem sinal (+127 com +1).

Nem todos os processadores oferecem essas facilidades, portanto sempre devemos observar a convenção ao iniciar com um processador novo.

c. Constantes e tipagem assembly

Como visto nos laboratórios de assembly, as instruções de formato I (como *addi*) só podem incluir uma constante de 12 bits; quando precisamos de constantes maiores, precisamos utilizar o formato U para carregar os demais 20 bits (com a instrução *lui*). Alternativamente, utilizamos a pseudoinstrução *li* para carregar qualquer constante e o montador do RARS vai se encarregar de utilizar um dos dois mecanismos, de acordo com a necessidade.

O assembly, no entanto, faz a escolha de expressar constantes decimais sempre como *signed*, ou seja, assume que um *addi* (p.ex.) deve ter valores entre -2^{11} e $+2^{11}-1$, i.e. $-2048 \leq cte \leq 2047$, o que determina a semântica

de que sempre haverá extensão de sinal. Isso pode parecer estranho, talvez porque na linguagem C nós podemos especificar a interpretação de uma constante com *signed* ou *unsigned*... Então com 12 bits, *se fossem unsigned*, poderíamos expressar uma constante como 3000_{10} .

No RARS, tal constante 3000 é considerada como necessitando de mais de 12 bits, porque as constantes normalmente são sinalizadas. Portanto precisamos carregá-la com duas instruções, expressando-a em 32 bits.

Apesar disto, por conveniência, podemos especificar valores de 32 bits em hexadecimal como se fossem binários, sem interpretar o sinal.⁷ Desta forma, os bits $1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1101_2 = \text{FFFF FFFD}_{16}$ podem ser carregados num registrador, e.g. `s2`, tanto com a instrução `li s2, 0xFFFFFFFD` quanto com a instrução `li s2, -3`.

Note-se que outros montadores e assemblys podem (e de fato fazem) escolhas diferentes.

d. Deslocamentos de bit

São as operações de *shift* e, dependendo do processador, *rotate*⁸. Os *shifts left* e *right* equivalem aproximadamente aos operadores `<<` e `>>` da linguagem C.

Para números *unsigned*, eles equivalem a multiplicar um número por 2^n ou dividir por 2^n , sendo n o número de bits; os bits faltantes serão preenchidos com zero. Sempre que houver uma potência de 2 numa multiplicação, um bom compilador usará as instruções nativas de *shift* do processador.

Infelizmente, com números *signed*, os resultados não são óbvios: o que obtemos se calcularmos $-83 \gg 2$? Depende do processador. Usualmente, no assembly, haverá uma instrução de deslocamento à direita que preenche com zero (chamado deslocamento “lógico”) e outra que preenche com o bit de sinal (o bit MSB, deslocamento “aritmético”), e a linguagem C não obriga o compilador a escolher um destes para os *signed*...⁹

Portanto, em 8 bits como sempre (facilita): $-83 \gg 2 \Rightarrow 10101101_2 \gg 2$; no caso lógico, dá $00101011_2 = 53$, e o sinal mudou! Já no caso aritmético, dá $11101011_2 = -21$, que apesar de não mudar o sinal, *arredonda a divisão para cima!* No caso positivo ele faz $83 \gg 2 = 20$, que é exatamente o mesmo que $83/4$.

⁷Também podemos utilizar o operador de negação unário (“-”) para fazer a negação de um número expresso em hexadecimal, mas aí parece preciosismo entender isso – vale pra quem for trabalhar programando em assembly, e ninguém mais. Enfim, dá pra fazer `li s2, -0x3`.

⁸As de rotação são idênticas às de deslocamento, exceto pelo fato de ao invés de descartarem o bit da ponta, o reintroduzem do outro lado. Rotacionar 10010000_2 para a esquerda resulta em 00100000_2 , por exemplo.

⁹A bíblia C90 nos diz: “Right shifting a signed quantity will fill with bit signs (“arithmetic shift”) on some machines and with 0-bits (“logical shift”) on others.” (Kernighan p.97)

3 A OBSOLETA SINAL-MAGNITUDE

Operações feitas em complemento de dois utilizam *os mesmos circuitos* do que as operações de números não-sinalizados (pois em módulo o resultado é o mesmo), o que é bastante conveniente. Estes circuitos são simples e podem ser otimizados para serem mais rápidos.

Uma outra opção que parece simples é utilizar a representação **sinal-magnitude**: o bit mais significativo (MSB) indica o sinal + ou - e o restante dos menos significativos representam a magnitude ou o número em si. Por exemplo, o número de 8 bits $\underline{1}0000110$ significa -6_{10} (o 1 à esquerda é o sinal negativo).

Dois inconvenientes: temos +0 e -0, o que enche o saco, e a circuitaria pra fazer as contas precisa levar em conta o sinal o tempo inteiro (comparar os sinais dos operandos pra descobrir se tem que somar ou subtrair, por exemplo). Portanto só computadores históricos adotam isso.

Dependendo do masoquismo de cada um, podemos usar **complemento de 1**¹⁰ se for o caso; basta inverter todos os bits para achar o negativo. Novamente temos -0 (em 8 bits, 11111111), e novamente as operações precisam de hardware adicional; precisamos ajustar as somas somando 1 se houver carry, por exemplo. Tente somar $13 + (-4)$, dá $00001101_2 + 11111011_2$, que resulta em $(1)00001000_2 = 8$, que deve ser incrementado para $00001001_2 = 9$.

À parte destas opções sensatas porém obsoletas, sempre podemos tentar usar base ternária balanceada ou base menos dez (negadecimal), ou coisas ainda mais exóticas.¹¹

4 EXERCÍCIOS

1. Como é representado o número -84 em complemento de 2 com 8 bits?
2. Qual é a diferença entre Carry e Overflow?
3. Suponha que o processador que estamos trabalhando é de 6 bits. Vamos somar os números binários $101110_2 + 100100_2$. a) Qual é o resultado? b) Qual é o significado deste número? Ou seja, qual o valor decimal dele? c) Houve carry? d) Houve overflow?
4. Quais são as operações aritméticas que são idênticas tanto para *signed* quanto para *unsigned*? (Ou seja, produzem o mesmo resultado independente de estarem usando complemento de 2 ou número binário). Quais são as operações que são diferentes?

¹⁰“Quanto falta para 1, em cada bit”; se o bit é zero, falta 1 para somar 1; se o bit é um, falta 0 para somar 1.

¹¹Usando $2i$ como base, por exemplo, sendo $i = \sqrt{-1}$, e dígitos de 0 a 3, o “quater-imaginário”. Consulte o Donald Knuth Vol. 2, Seminumerical Algorithms.

5. O que o C90 (C padrão ANSI 1990) vai fazer com o código abaixo? Há situações que podem gerar um problema? Se sim, quais?

```
unsigned char x,z;
signed char y,w;
x = f1();
y = f2();

if(y<x)
    z = w = y;
else
    w = z = x;
```

Se precisar entender conversões de tipo em C, consulte o Kernighan.

¹² O tipo char tem sempre 8 bits (mas em microcontroladores e C99 provavelmente seria melhor usar uint8_t e sint8_t, se for esse o caso).

6. Vá no *Compiler explorer* (<https://godbolt.org/>) e compile o programa abaixo em C selecionando como compilador-target o “RISC-V rv32gc clang (trunk)” e insira -O3 na caixa Compiler Options ao lado (é hífen – letra Ó maiúscula – 3), para limpar o código.

```
int main()
{
    int n1 = 2012345, n2;
    n2 = n1+n1;
    return n2;
}
```

Explique as constantes 983 e -1678 da listagem (O que elas representam? Como o número 2012345 da listagem original produz elas?).

7. Considerando um número em complemento de 2 com 6 bits, qual será o resultado final das somas consecutivas 30+30+30+30+30?
8. Considerando um número binário (*unsigned*) de 6 bits, qual será o resultado final das somas consecutivas 50+50+50+50?
9. Na representação sinal-magnitude, temos *dois zeros*: +0 e -0. Não esquecendo disso, construa com portas lógicas a operação da ULA que

¹²Kernighan & Ritchie “The C Programming Language Second Edition” ou, no Brasil, “C A Linguagem de Programação Padrão ANSI”. Nos apêndices deste livro encontra-se uma seção com as regras de conversão.

compara dois números e determina se o primeiro é maior ou igual ao segundo. Prove que estender o sinal é equivalente a copiar o bit MSB para todos os bits à esquerda. Se quiser facilitar, assuma um número fixo de bits, como 8, por exemplo.

10. Como podemos detectar overflow ou carry usando a linguagem C99 padrão?
11. Como podemos detectar overflow ou carry usando a linguagem assembly RISC-V?